# УЧЕБНИК ПО БАЗАМ ДАННЫХ

SQLite и PHP через PDO

Этот учебник — практическое руководство по работе с базой данных SQLite из PHP через стандартный интерфейс PDO. SQLite выбрана как самая простая СУБД для обучения: не требует установки сервера, работает с одним файлом, поддерживает большую часть SQL и идеально подходит для прототипов, конфигов и небольших веб-приложений.

Все примеры рассчитаны на PHP 8.x с расширением PDO_SQLITE (по умолчанию входит в дистрибутив). Никаких внешних серверов — достаточно открыть пример в браузере через встроенный сервер PHP: php -S localhost:8000.

---

## ОГЛАВЛЕНИЕ


- Глава 1. Введение в SQLite
- Глава 2. Создание таблиц
- Глава 3. Подключение через PDO
- Глава 4. CRUD: вставка, чтение, обновление, удаление
- Глава 5. Связи таблиц и внешние ключи
- Глава 6. JOIN и агрегатные функции
- Глава 7. Транзакции и ACID
- Глава 8. Пагинация, фильтрация, сортировка
- Глава 9. Безопасность
- Глава 10. ООП-обёртки (Repository/Model)
- Глава 11. Миграции схемы БД
- Глава 12. Практические задачи и итоги


---

## ГЛАВА 1. ВВЕДЕНИЕ В SQLITE

## 1.1. Что такое SQLite

SQLite — это встроенная реляционная СУБД. В отличие от MySQL или PostgreSQL, которые работают как отдельный сервер (отдельный процесс, слушает порт), SQLite встроена прямо в приложение. База данных хранится в одном файле на диске, а доступ к ней идёт через библиотеку, скомпилированную в PHP.

Это даёт несколько важных преимуществ. Ноль настройки: не нужно создавать пользователей, настраивать права, открывать порты. Один файл: вся база — это один .sqlite или .db файл, который можно скопировать, передать, забэкапить как обычный файл. Стандартный SQL: поддерживает большую часть SQL-92 плюс расширения.

## 1.2. Где применяется SQLite

SQLite — самая распространённая СУБД в мире, хотя многие этого не подозревают. Она встроена в:

- Все мобильные ОС — Android, iOS
- Все браузеры — Chrome, Firefox, Safari
- Мессенджеры — WhatsApp, Telegram хранят историю в SQLite
- Skype, Dropbox, Adobe Lightroom
- Конфигурацию многих embedded-устройств
- Кэши CMS и фреймворков
- Малые и средние сайты с невысокой нагрузкой

Для веб-разработки SQLite идеальна для прототипов, небольших сайтов (визитки, блоги, корпоративные страницы), админок с малой нагрузкой, тестовых окружений. Для высоконагруженных проектов с тысячами одновременных записей лучше MySQL/PostgreSQL — SQLite блокирует весь файл при записи.

## 1.3. SQLite vs MySQL — кратко

Главные отличия:

- SQLite: один файл, нет сервера, нет пользователей
- MySQL: отдельный сервер, пользователи и привилегии
- SQLite: один писатель, много читателей
- MySQL: много писателей, полноценные блокировки строк
- SQLite: типы динамические (можно вставить строку в INTEGER)
- MySQL: строгая типизация колонок
- SQLite: типы INTEGER, TEXT, REAL, BLOB, NUMERIC
- MySQL: INT, VARCHAR, TEXT, DATETIME, DECIMAL и т.д.

SQLite не умеет RIGHT JOIN и FULL OUTER JOIN (хотя в свежих версиях FULL OUTER уже добавлен). Зато поддерживает внешние ключи, транзакции и триггеры — почти как «взрослая» СУБД. Для обучения и небольших проектов этого более чем достаточно.

## 1.4. Проверка расширения PDO_SQLITE

Перед началом проверим, что PHP умеет работать с SQLite. Создайте файл phpinfo.php в корне проекта и откройте в браузере:

```
<?php
phpinfo();
```

Найдите раздел PDO — должны увидеть pdo_sqlite с supported. Если нет, установите расширение:

```
# Ubuntu/Debian
apt install php-sqlite3
systemctl restart php8.3-fpm  # или apache2

# Проверка из CLI:
php -m | grep sqlite
# pdo_sqlite
# sqlite3
```

## 1.5. CLI-клиент sqlite3

Помимо работы через PHP, у SQLite есть CLI-клиент sqlite3. Установка:

```
# Ubuntu/Debian
apt install sqlite3

# Создать базу (создаётся файл database.sqlite)
sqlite3 database.sqlite

# Запустить REPL:
sqlite> .tables
sqlite> .schema users
sqlite> SELECT * FROM users;
sqlite> .quit
```

Полезные dot-команды:

- .tables — список таблиц
- .schema — схема всех таблиц
- .schema users — схема одной таблицы
- .headers on — показывать заголовки колонок

- .mode column — выровнять колонки
- .read script.sql — выполнить SQL из файла
- .dump — экспорт всей базы в SQL-текст

CLI удобен для быстрой проверки структуры и данных без написания PHP-кода. Используйте его для отладки.

---

# ГЛАВА 2. СОЗДАНИЕ ТАБЛИЦ

## 2.1. Типы данных SQLite

SQLite использует систему типов с динамической типизацией. В отличие от MySQL, где тип колонки строго определяет, какие данные можно записать, SQLite применяет «аффинность типов» (type affinity) — предпочтительный тип, который пытается применить к значениям.

Пять основных аффинностей:

- INTEGER — целое число, 1-8 байт
- TEXT — строка в UTF-8, UTF-16 или UTF-16BE
- REAL — число с плавающей точкой, 8 байт
- BLOB — бинарные данные как есть
- NUMERIC — попытка конвертации в INTEGER или REAL

Объявления типов MySQL VARCHAR(255), INT, DATETIME в SQLite сводятся к одной из этих аффинностей. VARCHAR(50) и TEXT обе станут TEXT. Это значит, что SQLite не валидирует длину строк на уровне типа — это ответственность приложения.

Дата и время обычно хранят как TEXT в формате ISO 8601 (YYYY-MM-DD HH:MM:SS) или как INTEGER — Unix timestamp. Удобнее ISO 8601: он читаем в БД и сортируется лексикографически.

## 2.2. CREATE TABLE — синтаксис

Создание таблицы — оператор CREATE TABLE. Самый простой пример:

```
CREATE TABLE users (
    id       INTEGER PRIMARY KEY AUTOINCREMENT,
    name     TEXT NOT NULL,
    email    TEXT NOT NULL UNIQUE,
    age      INTEGER,
    created  TEXT DEFAULT CURRENT_TIMESTAMP
);
```

Разберём по частям:

- id INTEGER PRIMARY KEY AUTOINCREMENT — счётчик строк
- name TEXT NOT NULL — обязательное поле
- email TEXT NOT NULL UNIQUE — уникальный email
- age INTEGER — необязательное число
- created TEXT DEFAULT CURRENT_TIMESTAMP — текущее время

## 2.3. PRIMARY KEY и AUTOINCREMENT

PRIMARY KEY — первичный ключ, уникально идентифицирует строку в таблице. В SQLite колонка с PRIMARY KEY автоматически становится ROWID — внутренним идентификатором строки. Если тип INTEGER, то SQLite автоматически присваивает новое значение при вставке NULL.

AUTOINCREMENT меняет поведение: каждый новый id гарантированно больше любого из когда-либо существовавших, даже после удаления записей. Без AUTOINCREMENT SQLite может переиспользовать id удалённой строки. На практике AUTOINCREMENT нужен нечасто, но безопаснее.

## 2.4. NOT NULL, DEFAULT, UNIQUE

Ограничения (constraints) — это правила, которые БД проверяет при вставке и обновлении:

- NOT NULL — поле не может быть пустым (NULL)
- DEFAULT значение — значение по умолчанию
- DEFAULT CURRENT_TIMESTAMP — текущая дата/время
- UNIQUE — значение должно быть уникальным в таблице
- CHECK (условие) — произвольная проверка

```
CREATE TABLE products (
    id       INTEGER PRIMARY KEY AUTOINCREMENT,
    name     TEXT NOT NULL,
    price    REAL NOT NULL CHECK (price >= 0),
    quantity INTEGER NOT NULL DEFAULT 0,
    sku      TEXT UNIQUE,
    created  TEXT DEFAULT CURRENT_TIMESTAMP
);
```

Здесь CHECK (price >= 0) не даст вставить товар с отрицательной ценой, DEFAULT 0 выставит нулевое количество, если не указано, а UNIQUE на sku не допустит дубликатов артикулов.

### 2.5. Создание базы через PHP

Хотя CREATE TABLE можно выполнить через sqlite3 CLI, обычно схему создают скриптом при установке. Простейший пример на PHP:

```
<?php
$pdo = new PDO('sqlite:database.sqlite');
$pdo->setAttribute(PDO::ATTR_ERRMODE,
                   PDO::ERRMODE_EXCEPTION);

$sql = "CREATE TABLE IF NOT EXISTS users (
    id       INTEGER PRIMARY KEY AUTOINCREMENT,
    name     TEXT NOT NULL,
    email    TEXT NOT NULL UNIQUE,
    age      INTEGER,
    created  TEXT DEFAULT CURRENT_TIMESTAMP
)";

$pdo->exec($sql);
echo "Таблица создана";
?>
```

Файл database.sqlite создастся автоматически в текущей папке, если его не было. Конструкция IF NOT EXISTS позволяет безопасно перезапускать скрипт — таблица создастся только один раз, последующие запуски ничего не меняют.

---

# ГЛАВА 3. ПОДКЛЮЧЕНИЕ ЧЕРЕЗ PDO

## 3.1. Что такое PDO

PDO (PHP Data Objects) — это стандартный интерфейс доступа к базам данных в PHP. Он унифицирует работу с разными СУБД: одни и те же методы prepare(), execute(), fetch() работают и для SQLite, и для MySQL, и для PostgreSQL. Меняется только DSN — строка подключения.

Главные преимущества PDO перед устаревшими расширениями (mysqli, sqlite3):

- Подготовленные запросы «из коробки»
- Именованные параметры (:name), а не только ?
- Единый API для разных СУБД
- Обработка ошибок через исключения
- Поддержка транзакций на уровне API
- Несколько режимов выборки (FETCH_*)

## 3.2. DSN для SQLite

DSN (Data Source Name) — строка, описывающая подключение. Для SQLite формат простой:

```
'sqlite:путь/к/файлу.sqlite'

# Примеры:
'sqlite:database.sqlite'
'sqlite:/var/www/mysite/data/app.sqlite'
'sqlite::memory:'   # база в памяти, исчезает
                    # после завершения скрипта
```

База в памяти (:memory:) удобна для тестов: создаётся мгновенно, не оставляет файлов, изолирована между запросами. Не подходит для продакшена — данные не сохраняются между запусками скрипта.

## 3.3. Создание подключения

Подключение создаётся оператором new PDO(...). Сразу после создания рекомендуется задать режимы обработки ошибок и выборки:

```php
<?php
$pdo = new PDO('sqlite:database.sqlite');

# Режим ошибок: бросать исключение
$pdo->setAttribute(PDO::ATTR_ERRMODE,
                   PDO::ERRMODE_EXCEPTION);

# Режим выборки: ассоциативный массив
$pdo->setAttribute(PDO::ATTR_DEFAULT_FETCH_MODE,
                   PDO::FETCH_ASSOC);
?>
```

Без ATTR_ERRMODE = ERRMODE_EXCEPTION PDO молча игнорирует ошибки SQL — придётся проверять код возврата каждого метода вручную. С исключениями код чище и понятнее.

FETCH_ASSOC заставляет fetch() и fetchAll() возвращать ассоциативные массивы с ключами-именами колонок. По умолчанию PDO возвращает и числовые индексы, и имена — то есть дублирует данные. FETCH_ASSOC даёт чистый вывод.

## 3.4. Обработка ошибок подключения

Если файл недоступен для записи, PDO бросит PDOException. Оборачивайте создание подключения в try/catch:

```php
<?php
try {
    $pdo = new PDO('sqlite:database.sqlite');
    $pdo->setAttribute(PDO::ATTR_ERRMODE,
                       PDO::ERRMODE_EXCEPTION);
} catch (PDOException $e) {
    error_log($e->getMessage());
    die('Не удалось подключиться к БД');
}
?>
```

Подробное сообщение об ошибке логируйте в файл, пользователю показывайте общую фразу. Технические детали могут помочь злоумышленнику понять структуру вашей системы.

## 3.5. Общая функция подключения

Чтобы не дублировать код во всех скриптах, вынесите подключение в отдельный файл и подключайте через require_once. Типичная структура:

```php
<?php
# db.php — конфигурация подключения
function db(): PDO {
    static $pdo = null;
    if ($pdo === null) {
        $pdo = new PDO(
            'sqlite:' . __DIR__
                      . '/database.sqlite'
        );
        $pdo->setAttribute(
            PDO::ATTR_ERRMODE,
            PDO::ERRMODE_EXCEPTION
        );
        $pdo->setAttribute(
            PDO::ATTR_DEFAULT_FETCH_MODE,
            PDO::FETCH_ASSOC
        );
    }
    return $pdo;
}
?>
```

Использование в других скриптах:

```php
<?php
require_once 'db.php';

$users = db()->query('SELECT * FROM users')
             ->fetchAll();
?>
```

static $pdo = null внутри функции сохраняет объект PDO между вызовами. Это реализация паттерна Singleton в виде функции — одно подключение на весь запрос.

---

# ГЛАВА 4. CRUD: ВСТАВКА, ЧТЕНИЕ, ОБНОВЛЕНИЕ, УДАЛЕНИЕ

## 4.1. INSERT — вставка

Вставка одной записи — оператор INSERT INTO. С подготовленными запросами это выглядит так:

```
$sql = 'INSERT INTO users (name, email, age)
        VALUES (:name, :email, :age)';

$stmt = db()->prepare($sql);
$stmt->execute([
    ':name'  => 'Анна',
    ':email' => 'anna@example.com',
    ':age'   => 28,
]);

echo 'Новый id: ' . db()->lastInsertId();
```

Что здесь происходит. prepare() парсит SQL с плейсхолдерами (:name, :email, :age) и строит план выполнения. execute() подставляет значения и запускает запрос. PDO сам экранирует строки, преобразует типы — это гарантированная защита от SQL-инъекций.

Метод lastInsertId() возвращает id только что вставленной записи. Работает для колонок с AUTOINCREMENT или эквивалентных.

## 4.2. Сокращённая запись без имён

Можно использовать знаки вопроса ? вместо именованных параметров. Код короче, но менее читаем:

```
$sql = 'INSERT INTO users (name, email, age)
        VALUES (?, ?, ?)';

db()->prepare($sql)->execute([
    'Анна',
    'anna@example.com',
    28,
]);
```

Правило: если параметров больше трёх, используйте именованные плейсхолдеры. Если меньше — ? приемлемо. Именованные параметры можно передавать в любом порядке.

## 4.3. SELECT — чтение одной записи

Чтение одной строки — метод fetch(). Возвращает ассоциативный массив (при FETCH_ASSOC) или false, если записей нет:

```php
$stmt = db()->prepare(
    'SELECT * FROM users WHERE id = :id'
);
$stmt->execute([':id' => 42]);

$user = $stmt->fetch();

if ($user === false) {
    echo 'Пользователь не найден';
} else {
    echo $user['name'];  // Анна
    echo $user['email']; // anna@example.com
}
```

## 4.4. SELECT — чтение нескольких записей

Чтение всех подходящих строк — fetchAll(). Возвращает массив ассоциативных массивов:

```php
$stmt = db()->prepare(
    'SELECT * FROM users WHERE age >= :min
     ORDER BY name'
);
$stmt->execute([':min' => 18]);

foreach ($stmt->fetchAll() as $u) {
    echo $u['name'] . ' (' . $u['age'] . ")\n";
}
```

Для простых SELECT без пользовательских данных можно использовать query() — он выполняет SQL сразу:

```php
$users = db()->query(
    'SELECT * FROM users ORDER BY name'
)->fetchAll();
```

Если в запросе есть параметры от пользователя — всегда prepare + execute, никогда не query с конкатенацией. Это главное правило безопасности (глава 9).

## 4.5. fetchColumn — одно значение

Когда нужно достать одно значение (например, количество записей или одно поле), используйте fetchColumn():

```
$count = db()->query(
    'SELECT COUNT(*) FROM users'
)->fetchColumn();
echo "Всего: $count";

# Имя пользователя по id
$stmt = db()->prepare(
    'SELECT name FROM users WHERE id = ?'
);
$stmt->execute([42]);
$name = $stmt->fetchColumn();
```

fetchColumn() возвращает первую колонку первой строки. Если записей нет — false.

## 4.6. UPDATE — обновление

Обновление существующих записей — оператор UPDATE. С подготовленным запросом:

```
$stmt = db()->prepare(
    'UPDATE users
     SET name = :name, age = :age
     WHERE id = :id'
);
$stmt->execute([
    ':name' => 'Анна Петрова',
    ':age'  => 29,
    ':id'   => 42,
]);

echo 'Обновлено: ' . $stmt->rowCount() . ' строк';
```

Метод rowCount() возвращает количество затронутых строк. Важно: если UPDATE не изменил никаких значений (поля уже были такими), rowCount может вернуть 0 — это не ошибка.

## 4.7. DELETE — удаление

Удаление записей — оператор DELETE. Всегда с WHERE, иначе удалите все строки таблицы:

```
$stmt = db()->prepare(
    'DELETE FROM users WHERE id = :id'
);
$stmt->execute([':id' => 42]);

echo 'Удалено: ' . $stmt->rowCount();
```

В production часто делают мягкое удаление — помечают запись как удалённую через флаг deleted_at, но не стирают физически. Это позволяет восстановить данные и сохранить историю.

```
-- Добавить колонку для мягкого удаления
ALTER TABLE users
  ADD COLUMN deleted_at TEXT;

-- "Удаление" — установка времени
UPDATE users SET deleted_at = CURRENT_TIMESTAMP
  WHERE id = 42;

-- Чтение только активных
SELECT * FROM users WHERE deleted_at IS NULL;
```

## 4.8. Шаблон CRUD-операции в PHP

Сводный пример всех операций:

```
<?php
require_once 'db.php';

# CREATE
db()->prepare(
    'INSERT INTO users (name, email, age)
     VALUES (?, ?, ?)'
)->execute(['Анна', 'a@e.com', 28]);
$newId = db()->lastInsertId();

# READ one
$u = db()->prepare('SELECT * FROM users WHERE id = ?');
$u->execute([$newId]);
$user = $u->fetch();

# READ all
$all = db()->query('SELECT * FROM users ORDER BY id')
           ->fetchAll();

# UPDATE
db()->prepare(
    'UPDATE users SET age = ? WHERE id = ?'
)->execute([29, $newId]);

# DELETE
db()->prepare('DELETE FROM users WHERE id = ?')
    ->execute([$newId]);
?>
```

# ГЛАВА 5. СВЯЗИ ТАБЛИЦ И ВНЕШНИЕ КЛЮЧИ

## 5.1. Типы связей

Между таблицами бывают три типа связей:

- 1:1 (один-к-одному) — редко, напр. user → profile
- 1:N (один-ко-многим) — пользователь → его посты
- N:M (много-ко-многим) — посты ↔ теги через связующую таблицу

Связь реализуется через внешний ключ (FOREIGN KEY) — колонку, ссылающуюся на PRIMARY KEY другой таблицы. В 1:N ключ ставится на стороне «много» (у постов есть user_id, не наоборот).

## 5.2. Пример схемы блога

Создадим схему небольшого блога: пользователи пишут посты, к постам оставляют комментарии. Посты помечаются тегами (N:M):

```
-- Таблица пользователей
CREATE TABLE users (
    id       INTEGER PRIMARY KEY AUTOINCREMENT,
    name     TEXT NOT NULL,
    email    TEXT NOT NULL UNIQUE,
    created  TEXT DEFAULT CURRENT_TIMESTAMP
);

-- Таблица постов (1:N: у поста один автор,
-- у автора много постов)
CREATE TABLE posts (
    id       INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id  INTEGER NOT NULL,
    title    TEXT NOT NULL,
    body     TEXT NOT NULL,
    created  TEXT DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id)
        REFERENCES users(id)
        ON DELETE CASCADE
);

-- Таблица комментариев (1:N к постам)
CREATE TABLE comments (
    id       INTEGER PRIMARY KEY AUTOINCREMENT,
    post_id  INTEGER NOT NULL,
    author   TEXT NOT NULL,
    body     TEXT NOT NULL,
    created  TEXT DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (post_id)
        REFERENCES posts(id)
        ON DELETE CASCADE
);

-- Таблица тегов
CREATE TABLE tags (
    id   INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL UNIQUE
);

-- Связующая таблица для N:M
CREATE TABLE post_tags (
    post_id INTEGER NOT NULL,
    tag_id  INTEGER NOT NULL,
    PRIMARY KEY (post_id, tag_id),
    FOREIGN KEY (post_id) REFERENCES posts(id)
        ON DELETE CASCADE,
    FOREIGN KEY (tag_id)  REFERENCES tags(id)
        ON DELETE CASCADE
);
```

## 5.3. Включение внешних ключей

Важно: SQLite по умолчанию НЕ проверяет внешние ключи. Это сделано для обратной совместимости со старыми версиями. Чтобы внешние ключи заработали, нужно включить их при каждом подключении:

```
$pdo = new PDO('sqlite:blog.sqlite');
$pdo->setAttribute(PDO::ATTR_ERRMODE,
                   PDO::ERRMODE_EXCEPTION);

# Включить проверку внешних ключей
$pdo->exec('PRAGMA foreign_keys = ON');
```

PRAGMA — это специальная команда SQLite для управления режимами работы. foreign_keys = ON включает проверку FOREIGN KEY ограничений. Без неё можно вставить post с user_id = 99999, даже если такого пользователя нет.

Дополните db.php из главы 3 строкой $pdo->exec('PRAGMA foreign_keys = ON') сразу после создания подключения.

### 5.4. ON DELETE — действие при удалении

При удалении родительской записи можно автоматически обработать связанные:

- CASCADE — удалить все зависимые записи
- SET NULL — поставить foreign key в NULL (колонка должна быть nullable)
- RESTRICT — запретить удаление, пока есть зависимости
- NO ACTION — то же, что RESTRICT (по умолчанию)

В нашей схеме блога: при удалении пользователя CASCADE автоматически удалит все его посты, а вместе с постами — и комментарии, и связи тегов (CASCADE у post_tags и comments). Полное удаление со всем наследием одной командой DELETE FROM users WHERE id = 5.

### 5.5. ON UPDATE — действие при изменении ключа

Аналогично ON DELETE работает ON UPDATE — действие при изменении значения PRIMARY KEY родительской записи. На практике почти не используется, потому что PRIMARY KEY обычно неизменен (AUTOINCREMENT id). Но если у вас естественный ключ (например, ISO-код страны), то ON UPDATE CASCADE обновит все ссылки автоматически.

---

# ГЛАВА 6. JOIN И АГРЕГАТНЫЕ ФУНКЦИИ

## 6.1. INNER JOIN — пересечение

JOIN объединяет данные из нескольких таблиц по условию. INNER JOIN возвращает только те строки, у которых есть совпадение в обеих таблицах.

```
-- Все посты с именами авторов
SELECT posts.id, posts.title,
       users.name AS author
FROM posts
INNER JOIN users ON posts.user_id = users.id
ORDER BY posts.created DESC;
```

Результат — таблица с колонками из posts и users. AS author задаёт псевдоним колонке users.name, чтобы не путать с другими name колонками.

## 6.2. LEFT JOIN — все левые

LEFT JOIN возвращает все строки из левой таблицы, даже если в правой нет совпадения. Несовпадающие колонки будут NULL. Это удобно, когда нужно вывести всё, включая пустые связи:

```
-- Все пользователи и их посты,
-- даже если постов нет
SELECT users.name,
       COUNT(posts.id) AS post_count
FROM users
LEFT JOIN posts ON posts.user_id = users.id
GROUP BY users.id;
```

Если бы был INNER JOIN, пользователи без постов вообще бы не попали в результат. LEFT JOIN гарантирует их присутствие с post_count = 0.

## 6.3. RIGHT JOIN и SQLite

SQLite исторически не поддерживал RIGHT JOIN (полная поддержка появилась только в версии 3.39+ от 2022 года). На старых версиях RIGHT JOIN нужно эмулировать: поменять таблицы местами и использовать LEFT JOIN. В современной версии можно писать как обычно:

```
-- Все посты, даже без комментариев
SELECT posts.title,
       COUNT(comments.id) AS c_count
FROM comments
RIGHT JOIN posts ON comments.post_id = posts.id
GROUP BY posts.id;
```

Но правило хорошего тона: всегда используйте LEFT JOIN, меняя таблицы местами. Так код работает на любой версии SQLite и одинаково читается.

## 6.4. N:M — связь через связующую таблицу

Для выборки постов с тегами придётся делать два JOIN: один к post_tags, второй к tags:

```
-- Все посты с их тегами
SELECT posts.title,
       tags.name AS tag
FROM posts
INNER JOIN post_tags
    ON post_tags.post_id = posts.id
INNER JOIN tags
    ON tags.id = post_tags.tag_id
ORDER BY posts.id;
```

Если у поста три тега, он появится в результате трижды — по одной строке на каждый тег. Чтобы получить пост один раз со списком тегов через запятую, используйте GROUP_CONCAT:

```
SELECT posts.title,
       GROUP_CONCAT(tags.name, ', ') AS tags
FROM posts
LEFT JOIN post_tags
    ON post_tags.post_id = posts.id
LEFT JOIN tags
    ON tags.id = post_tags.tag_id
GROUP BY posts.id;
```

## 6.5. GROUP BY — группировка

GROUP BY группирует строки по одинаковым значениям указанных колонок. С агрегатными функциями (COUNT, SUM, AVG, MAX, MIN) позволяет получить статистику по группам:

```
-- Сколько постов у каждого автора
SELECT users.name,
       COUNT(posts.id) AS posts
FROM users
LEFT JOIN posts ON posts.user_id = users.id
GROUP BY users.id
ORDER BY posts DESC;
```

### 6.6. Агрегатные функции

Самые ходовые агрегатные функции:

- COUNT(*) — количество строк в группе
- COUNT(col) — количество непустых значений col
- SUM(col) — сумма значений
- AVG(col) — среднее арифметическое
- MAX(col) — максимальное значение
- MIN(col) — минимальное значение
- GROUP_CONCAT(col, sep) — склейка строк через разделитель

```
-- Статистика по постам
SELECT
    COUNT(*) AS total_posts,
    COUNT(DISTINCT user_id) AS authors,
    AVG(LENGTH(body)) AS avg_body_len,
    MAX(created) AS latest,
    MIN(created) AS earliest
FROM posts;
```

### 6.7. HAVING — фильтр после групп

WHERE фильтрует строки ДО группировки, HAVING — ПОСЛЕ. Поэтому HAVING может использовать агрегатные функции, а WHERE — нет:

```
-- Авторы с 5+ постами
SELECT users.name,
       COUNT(posts.id) AS cnt
FROM users
INNER JOIN posts ON posts.user_id = users.id
GROUP BY users.id
HAVING COUNT(posts.id) >= 5
ORDER BY cnt DESC;
```

---

# ГЛАВА 7. ТРАНЗАКЦИИ И ACID

### 7.1. Что такое транзакция

Транзакция — это группа операций, которые выполняются как единое целое: либо все успешно, либо никакая. Если посреди транзакции произошла ошибка, все предыдущие изменения отменяются (rollback). Если все успешны — изменения сохраняются (commit).

Зачем это нужно. Представьте перевод денег со счёта А на счёт Б. Это две операции: уменьшить А, увеличить Б. Если между ними упал сервер, получится, что деньги списаны, но не зачислены — клиент в ярости. Транзакция гарантирует, что такого не произойдёт.

## 7.2. ACID — свойства транзакций

ACID — акроним четырёх свойств, которые обеспечивают надёжность транзакций:

- A — Atomicity (атомарность): все или ничего
- C — Consistency (согласованность): БД остаётся в корректном состоянии
- I — Isolation (изоляция): параллельные транзакции не влияют друг на друга
- D — Durability (долговечность): после commit изменения сохраняются

Атомарность — главное свойство. Если транзакция из 5 операций упала на 4-й, первые 3 откатываются автоматически.

## 7.3. PDO API для транзакций

PDO предоставляет три метода:

- beginTransaction() — начать транзакцию
- commit() — подтвердить изменения
- rollBack() — откатить все изменения

Шаблон использования с try/catch:

```
$pdo = db();
try {
    $pdo->beginTransaction();

    # Списать деньги со счёта A
    $pdo->prepare(
        'UPDATE accounts SET balance = balance - ?
         WHERE id = ?'
    )->execute([$amount, $account_a]);

    # Зачислить на счёт Б
    $pdo->prepare(
        'UPDATE accounts SET balance = balance + ?
         WHERE id = ?'
    )->execute([$amount, $account_b]);

    $pdo->commit();
    echo 'Перевод выполнен';
} catch (PDOException $e) {
    $pdo->rollBack();
    error_log($e->getMessage());
    echo 'Ошибка перевода, деньги возвращены';
}
```

Важно: при ERRMODE_EXCEPTION PDO автоматически вызывает rollBack(), если произошло исключение, но только для активной транзакции. Явный rollBack() в catch — это перестраховка на случай, если исключение бросил не SQL, а ваш PHP-код между запросами.

## 7.4. Вложенные транзакции и savepoints

Строго говоря, транзакции не вкладываются: BEGIN внутри BEGIN в SQLite сразу делает commit. Но есть понятие savepoint — точки сохранения внутри большой транзакции, к которой можно откатиться частично:

```
$pdo->beginTransaction();

$pdo->exec('SAVEPOINT step1');
# ... операции ...
$pdo->exec('RELEASE SAVEPOINT step1');

$pdo->exec('SAVEPOINT step2');
# ... операции ...
# Что-то пошло не так на этом шаге
$pdo->exec('ROLLBACK TO SAVEPOINT step2');
# Откатились к шагу 2,
# но шаг 1 и всё остальное — на месте

$pdo->commit();
```

Savepoints полезны в сложных бизнес-процессах, где часть операций можно безопасно отменить, не откатывая всю транзакцию.

### 7.5. Уровни изоляции

Уровень изоляции определяет, насколько параллельные транзакции видят изменения друг друга. SQLite поддерживает два уровня через PRAGMA:

- SERIALIZABLE (по умолчанию в SQLite) — полная изоляция
- READ UNCOMMITTED — видно «грязные» незафиксированные данные

SQLite реализует SERIALIZABLE через блокировку файла базы: только одна транзакция записи одновременно. Это делает SQLite непригодным для высоконагруженных приложений с интенсивной записью (тысячи транзакций в секунду). Для учебных проектов и малонагруженных сайтов SERIALIZABLE — отличная гарантия целостности.

---

# ГЛАВА 8. ПАГИНАЦИЯ, ФИЛЬТРАЦИЯ, СОРТИРОВКА

### 8.1. LIMIT и OFFSET

Пагинация — это разбиение большого списка записей на страницы. Реализуется через LIMIT (сколько записей на странице) и OFFSET (сколько пропустить с начала):

```
-- Страница 3, по 20 записей на странице
SELECT * FROM posts
ORDER BY created DESC
LIMIT 20 OFFSET 40;
```

OFFSET = (номер_страницы - 1) × размер_страницы. Для третьей страницы по 20 записей это (3-1) × 20 = 40.

### 8.2. Подсчёт общего числа

Чтобы показать «Страница 3 из 27», нужно знать общее число записей. Простейший способ — отдельный запрос COUNT:

```
$total = db()->query(
    'SELECT COUNT(*) FROM posts'
)->fetchColumn();

$perPage = 20;
$pages = (int) ceil($total / $perPage);
echo "Страница $page из $pages";
```

## 8.3. Шаблон пагинатора

Соберём всё вместе в функцию, возвращающую данные страницы и метаданные для вывода навигации:

```
<?php
function paginate(string $table,
                  int   $page,
                  int   $perPage = 20,
                  string $order = 'id DESC',
                  string $where = ''): array {
    $pdo = db();

    # WHERE безопасно вставляем только как
    # статичную строку, не пользовательский ввод
    $w = $where ? "WHERE $where" : '';

    $total = $pdo->query(
        "SELECT COUNT(*) FROM $table $w"
    )->fetchColumn();

    $pages = max(1, (int) ceil($total / $perPage));
    $page  = max(1, min($page, $pages));
    $offset = ($page - 1) * $perPage;

    $stmt = $pdo->prepare(
        "SELECT * FROM $table $w
         ORDER BY $order
         LIMIT $perPage OFFSET $offset"
    );
    $stmt->execute();
    $items = $stmt->fetchAll();

    return [
        'items'  => $items,
        'page'   => $page,
        'pages'  => $pages,
        'total'  => $total,
    ];
}
?>
```

ВАЖНО: в этом примере переменные $table, $order, $where вставлены в SQL напрямую. Это безопасно ТОЛЬКО если они приходят из вашего кода (как имена таблиц или белый список сортировок), а НЕ от пользователя. Пользовательский

ввод (значения фильтров) должен идти через плейсхолдеры — см. главу 9.

Использование:

```
$result = paginate('posts',
                   page: $_GET['page'] ?? 1,
                   perPage: 10);

foreach ($result['items'] as $post) {
    echo '<h2>' . htmlspecialchars($post['title'])
         . '</h2>';
}

echo "Страница {$result['page']}
      из {$result['pages']}";
```

## 8.4. WHERE — фильтрация

WHERE сужает выборку. Операторы сравнения: =, <>, <, >, <=, >=, BETWEEN, IN, LIKE:

```
-- Пользователи от 18 до 65 лет
SELECT * FROM users
WHERE age BETWEEN 18 AND 65;

-- Имена на букву A
SELECT * FROM users
WHERE name LIKE 'A%';

-- Записи из списка id
SELECT * FROM users
WHERE id IN (1, 5, 12, 18);

-- Сложное условие
SELECT * FROM users
WHERE age >= 18
  AND (city = 'Москва' OR city = 'СПб')
  AND email IS NOT NULL;
```

С пользовательским вводом — через плейсхолдеры:

```
$stmt = db()->prepare(
    'SELECT * FROM users
     WHERE age >= :min AND city = :city'
);
$stmt->execute([
    ':min'  => $_GET['min_age'] ?? 0,
    ':city' => $_GET['city']   ?? '',
]);
$rows = $stmt->fetchAll();
```

### 8.5. ORDER BY — сортировка

ORDER BY сортирует результат по одной или нескольким колонкам. ASC — по возрастанию (по умолчанию), DESC — по убыванию:

```
-- Сначала новые посты
SELECT * FROM posts
ORDER BY created DESC;

-- По имени, потом по возрасту
SELECT * FROM users
ORDER BY name ASC, age DESC;
```

Если сортировка выбирается пользователем, всегда валидируйте имя колонки по белому списку:

```
$allowed = ['name', 'email', 'created'];
$sort = $_GET['sort'] ?? 'name';
if (!in_array($sort, $allowed)) {
    $sort = 'name';
}
$dir = strtolower($_GET['dir'] ?? 'asc');
if (!in_array($dir, ['asc', 'desc'])) {
    $dir = 'asc';
}
$sql = "SELECT * FROM users
        ORDER BY $sort $dir";
$rows = db()->query($sql)->fetchAll();
```

Имя колонки и направление нельзя передавать через плейсхолдер — PDO их не подставляет в идентификаторы. Поэтому обязательна проверка по белому списку (in_array).

---

# ГЛАВА 9. БЕЗОПАСНОСТЬ

### 9.1. SQL-инъекция — пример атаки

SQL-инъекция — это когда пользовательский ввод попадает в SQL без экранирования, и злоумышленник меняет логику запроса. Классический пример:

```
$id = $_GET['id'];
# ОПАСНО — никогда так не пишите
$sql = "SELECT * FROM users WHERE id = $id";
$user = db()->query($sql)->fetch();
```

Если злоумышленник передаст id = 1 OR 1=1, итоговый SQL станет:

```
SELECT * FROM users WHERE id = 1 OR 1=1
```

Условие 1=1 всегда истинно, поэтому запрос вернёт ВСЕХ пользователей. Хуже — id = 1; DROP TABLE users удалит таблицу целиком.

## 9.2. Подготовленные запросы — защита

Подготовленные запросы (prepared statements) полностью предотвращают SQL-инъекции. Идея: SQL и данные передаются отдельно. БД сначала парсит SQL и строит план выполнения, потом подставляет значения как параметры, а не как часть SQL.

```
# Безопасно
$stmt = db()->prepare(
    'SELECT * FROM users WHERE id = :id'
);
$stmt->execute([':id' => $_GET['id']]);
$user = $stmt->fetch();
```

Даже если $_GET['id'] = '1 OR 1=1', это будет передано как строковое значение, а не как SQL. Запрос просто не найдёт пользователя с таким id.

Правило: любые данные, приходящие от пользователя ($_GET, $_POST, $_COOKIE, HTTP-заголовки), должны идти в запрос только через плейсхолдеры. Никогда через конкатенацию.

## 9.3. Идентификаторы нельзя параметризовать

PDO не позволяет использовать плейсхолдеры для имён таблиц, колонок, ключевых слов. Это ограничение SQL, а не PDO — плейсхолдеры работают только для значений.

```
# НЕРАБОТАЕТ
$stmt = db()->prepare(
    'SELECT * FROM :table WHERE id = :id'
);

# Так нельзя. Имя таблицы нужно
# подставлять как строку,
# обязательно проверив по белому списку:
```

```
$allowedTables = ['users', 'posts', 'comments'];
$table = $_GET['table'] ?? 'users';
if (!in_array($table, $allowedTables)) {
    throw new InvalidArgumentException('Bad table');
}
$sql = "SELECT * FROM $table WHERE id = ?";
$stmt = db()->prepare($sql);
$stmt->execute([$_GET['id']]);
```

## 9.4. Валидация входных данных

Даже с подготовленными запросами валидируйте пользовательский ввод. Подготовленный запрос защищает от инъекций, но не от логических ошибок: вставки отрицательного возраста, пустого email, текста вместо числа.

```
$age = $_POST['age'] ?? '';
if (!ctype_digit($age) || (int)$age < 0
                        || (int)$age > 150) {
    $errors[] = 'Возраст должен быть числом от 0 до 150';
}

$email = trim($_POST['email'] ?? '');
if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
    $errors[] = 'Некорректный email';
}

if ($errors) {
    # показать ошибки пользователю
} else {
    # выполнить INSERT через prepare
}
```

## 9.5. Где хранить файл базы

Файл SQLite (.sqlite, .db) должен быть недоступен через веб. Если кто-то откроет https://example.com/database.sqlite, он скачает всю базу целиком — с паролями, личными данными, всем.

Правильное место — вне корня сайта. Структура:

```
/var/www/mysite/
    public/         # корень сайта
        index.php
        .htaccess
    data/           # НЕ доступен через веб
        database.sqlite
    config/
        db.php       # содержит db()
```

В db.php путь указывается как __DIR__ . '/../data/database.sqlite'. Так файл остаётся внутри проекта, но вне public/, и Apache/Nginx его не отдадут.

Если нельзя вынести файл из public/, защитите его через .htaccess в Apache:

```
# public/.htaccess
<FilesMatch "\.(sqlite|db|sqlite3)$">
    Require all denied
</FilesMatch>
```

Или в Nginx:

```
location ~ \.(sqlite|db|sqlite3)$ {
    deny all;
}
```

### 9.6. Пароли — никогда в открытом виде

Пароли в БД хранятся только хэшированными. Функции PHP password_hash() и password_verify() — стандарт:

```
# Регистрация
$hash = password_hash($password, PASSWORD_DEFAULT);
db()->prepare(
    'INSERT INTO users (email, password_hash)
     VALUES (?, ?)'
)->execute([$email, $hash]);

# Вход
$stmt = db()->prepare(
    'SELECT password_hash FROM users WHERE email = ?'
);
$stmt->execute([$email]);
$row = $stmt->fetch();

if ($row &amp;amp;&amp;amp; password_verify($password, $row['password_hash'])) {
    echo 'Вход выполнен';
} else {
    echo 'Неверный email или пароль';
}
```

# ГЛАВА 10. ООП-ОБЁРТКИ (REPOSITORY/MODEL)

## 10.1. Зачем ООП-обёртка

Прямые вызовы PDO в каждом скрипте работают, но плохо масштабируются. Если вы захотите поменять СУБД или добавить кеширование, придётся редактировать десятки файлов. Обёртка над PDO решает эти проблемы: единая точка изменений, переиспользование, тестирование.

## 10.2. Класс Database (Singleton)

Первый уровень обёртки — класс для подключения, инкапсулирующий PDO:

```
<?php
class Database
{
    private static ?PDO $instance = null;

    public static function pdo(): PDO
    {
        if (self::$instance === null) {
            self::$instance = new PDO(
                'sqlite:' . __DIR__
                          . '/../data/database.sqlite'
            );
            self::$instance->setAttribute(
                PDO::ATTR_ERRMODE,
                PDO::ERRMODE_EXCEPTION
            );
            self::$instance->setAttribute(
                PDO::ATTR_DEFAULT_FETCH_MODE,
                PDO::FETCH_ASSOC
            );
            self::$instance->exec(
                'PRAGMA foreign_keys = ON'
            );
        }
        return self::$instance;
    }
}
?>
```

## 10.3. Базовый класс Model

Второй уровень — абстрактный класс Model, реализующий стандартные операции CRUD. Конкретные модели (User, Post) наследуют его и задают имя таблицы:

```php
<?php
abstract class Model
{
    protected PDO $pdo;
    protected string $table;

    public function __construct()
    {
        $this->pdo = Database::pdo();
    }

    public function find(int $id): ?array
    {
        $stmt = $this->pdo->prepare(
            "SELECT * FROM {$this->table}
             WHERE id = ?"
        );
        $stmt->execute([$id]);
        $row = $stmt->fetch();
        return $row === false ? null : $row;
    }

    public function findAll(
        int $limit = 100,
        int $offset = 0
    ): array {
        $stmt = $this->pdo->prepare(
            "SELECT * FROM {$this->table}
             LIMIT ? OFFSET ?"
        );
        $stmt->execute([$limit, $offset]);
        return $stmt->fetchAll();
    }

    public function delete(int $id): bool
    {
        $stmt = $this->pdo->prepare(
            "DELETE FROM {$this->table}
             WHERE id = ?"
        );
        return $stmt->execute([$id]);
    }
}
?>
```

## 10.4. Конкретная модель User

Наследуем Model, задаём имя таблицы. Можно добавить специфичные методы (например, поиск по email):

```php
<?php
class User extends Model
{
    protected string $table = 'users';

    public function findByEmail(string $email): ?array
    {
        $stmt = $this->pdo->prepare(
            'SELECT * FROM users
             WHERE email = ?'
        );
        $stmt->execute([$email]);
        $row = $stmt->fetch();
        return $row === false ? null : $row;
    }

    public function create(
        string $name,
        string $email,
        string $passwordHash
    ): int {
        $stmt = $this->pdo->prepare(
            'INSERT INTO users
                (name, email, password_hash)
             VALUES (?, ?, ?)'
        );
        $stmt->execute([$name, $email, $passwordHash]);
        return (int) $this->pdo->lastInsertId();
    }
}
?>
```

## 10.5. Использование

В контроллерах код становится чистым и читаемым:

```php
$users = new User();

# Найти по id
$user = $users->find(42);
echo $user['name'];

# Найти по email
$anna = $users->findByEmail('anna@example.com');

# Создать
$id = $users->create(
    'Олег',
    'oleg@example.com',
    password_hash('secret', PASSWORD_DEFAULT)
);

# Удалить
$users->delete($id);
```

Если завтра решите перейти с SQLite на MySQL, поменяете только класс Database — весь остальной код останется без изменений. В этом сила абстракции.

---

# ГЛАВА 11. МИГРАЦИИ СХЕМЫ БД

## 11.1. Зачем нужны миграции

Когда вы впервые создаёте базу, схема проста — пара CREATE TABLE. Но проект растёт: добавляются новые поля, таблицы, индексы. Через год у вас в production одна версия схемы, у коллеги на компьютере — другая, на тестовом сервере — третья.

Миграции решают эту проблему. Это версионированные SQL-скрипты, которые применяются последовательно. Каждый раз, когда меняется схема, вы создаёте новый файл с номером версии, а система миграций выполняет только то, что ещё не применено.

## 11.2. Структура папок

Типичная структура:

```
migrations/
    001_create_users.sql
    002_add_age_to_users.sql
    003_create_posts_and_comments.sql
    004_add_index_on_users_email.sql
    005_add_soft_delete_to_posts.sql
migrator.php        # скрипт применения миграций
```

## 11.3. Примеры файлов миграций

Каждый файл — SQL-скрипт. Именование с нумерацией обеспечивает правильный порядок применения:

```
-- 001_create_users.sql
CREATE TABLE users (
    id       INTEGER PRIMARY KEY AUTOINCREMENT,
    name     TEXT NOT NULL,
    email    TEXT NOT NULL UNIQUE,
    created  TEXT DEFAULT CURRENT_TIMESTAMP
);
```

```
-- 002_add_age_to_users.sql
ALTER TABLE users ADD COLUMN age INTEGER;
```

```
-- 004_add_index_on_users_email.sql
CREATE UNIQUE INDEX idx_users_email
    ON users(email);
```

## 11.4. Скрипт Migrator

Скрипт применяет все миграции, которые ещё не выполнены. Для отслеживания использует вспомогательную таблицу schema_migrations:

```php
<?php
# migrator.php
require_once 'db.php';

$pdo = db();

# Создать таблицу отслеживания,
# если её ещё нет
$pdo->exec(
    'CREATE TABLE IF NOT EXISTS schema_migrations (
        filename TEXT PRIMARY KEY,
        applied  TEXT DEFAULT CURRENT_TIMESTAMP
    )'
);

# Получить уже применённые
$applied = $pdo->query(
    'SELECT filename FROM schema_migrations'
)->fetchAll(PDO::FETCH_COLUMN);

# Найти все миграции
$files = glob(__DIR__
    . '/migrations/*.sql');
sort($files);

foreach ($files as $file) {
    $name = basename($file);
    if (in_array($name, $applied)) {
        continue;  # уже применена
    }

    echo "Применяется: $name\n";
    $sql = file_get_contents($file);

    try {
        $pdo->beginTransaction();
        $pdo->exec($sql);
        $pdo->prepare(
            'INSERT INTO schema_migrations
                (filename) VALUES (?)'
        )->execute([$name]);
        $pdo->commit();
        echo "  OK\n";
    } catch (PDOException $e) {
        $pdo->rollBack();
        echo '  ОШИБКА: ' . $e->getMessage()
             . "\n";
        exit(1);
    }
}

echo "Все миграции применены\n";
?>
```

Запуск из командной строки:

```
php migrator.php
```

### 11.5. Прямые и обратные миграции

В больших проектах принято писать пары: UP (применить) и DOWN (откатить). Это позволяет вернуться к предыдущей версии, если что-то пошло не так. Для малых проектов достаточно только UP — откат делают восстановлением из бэкапа.

Идея Down-миграции для ALTER TABLE users ADD COLUMN age — это ALTER TABLE users DROP COLUMN age. Но в SQLite до недавнего времени DROP COLUMN не поддерживался — приходилось создавать новую таблицу, копировать данные, переименовывать. С версии 3.35.0 (2021) DROP COLUMN работает нативно.

---

# ГЛАВА 12. ПРАКТИЧЕСКИЕ ЗАДАЧИ И ИТОГИ

### 12.1. Задача 1: простой гостевой блок

Создайте гостевую книгу: пользователь вводит имя и сообщение, они сохраняются в БД, ниже выводятся последние 10. Решение:

```php
<?php
require_once 'db.php';
$pdo = db();

# Создать таблицу при первом запуске
$pdo->exec(
    'CREATE TABLE IF NOT EXISTS guestbook (
        id      INTEGER PRIMARY KEY AUTOINCREMENT,
        name    TEXT NOT NULL,
        message TEXT NOT NULL,
        created TEXT DEFAULT CURRENT_TIMESTAMP
    )'
);

# Обработка POST
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $name = trim($_POST['name'] ?? '');
    $msg  = trim($_POST['message'] ?? '');
    if ($name !== '' &amp;amp;&amp;amp; $msg !== '') {
        $pdo->prepare(
            'INSERT INTO guestbook (name, message)
             VALUES (?, ?)'
        )->execute([$name, $msg]);
        header('Location: /');
        exit;
    }
}

# Вывод последних 10
$entries = $pdo->query(
    'SELECT * FROM guestbook
     ORDER BY id DESC LIMIT 10'
)->fetchAll();
?>

<!-- HTML форма и вывод -->
<form method="post">
  <input name="name" placeholder="Имя"><br>
  <textarea name="message"></textarea><br>
  <button>Отправить</button>
</form>

<?php foreach ($entries as $e): ?>
  <article>
    <b><?= htmlspecialchars($e['name']) ?></b>
    <time><?= $e['created'] ?></time>
    <p><?= nl2br(htmlspecialchars($e['message'])) ?></p>
  </article>
<?php endforeach; ?>
```

## 12.2. Задача 2: импорт CSV в SQLite

Импортируйте users.csv в таблицу users. Решение:

```php
<?php
require_once 'db.php';
$pdo = db();

$pdo->exec(
    'CREATE TABLE IF NOT EXISTS users (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT, email TEXT UNIQUE, age INTEGER
    )'
);

$stmt = $pdo->prepare(
    'INSERT INTO users (name, email, age)
     VALUES (?, ?, ?)'
);

$fh = fopen('users.csv', 'r');
$header = fgetcsv($fh);  # пропустить заголовок

$pdo->beginTransaction();
try {
    while (($row = fgetcsv($fh)) !== false) {
        $stmt->execute($row);
    }
    $pdo->commit();
    echo 'Импорт завершён';
} catch (PDOException $e) {
    $pdo->rollBack();
    echo 'Ошибка: ' . $e->getMessage();
}
fclose($fh);
?>
```

## 12.3. Задача 3: экспорт в JSON

Экспортируйте все посты в JSON с именем автора и количеством комментариев.
Решение:

```
<?php
require_once 'db.php';
header('Content-Type: application/json;'
      . ' charset=utf-8');

$rows = db()->query(
    'SELECT posts.title,
            posts.created,
            users.name AS author,
            COUNT(comments.id) AS comments
     FROM posts
     INNER JOIN users
         ON posts.user_id = users.id
     LEFT JOIN comments
         ON comments.post_id = posts.id
     GROUP BY posts.id
     ORDER BY posts.created DESC'
)->fetchAll();

echo json_encode($rows,
    JSON_UNESCAPED_UNICODE
    | JSON_PRETTY_PRINT);
?>
```

## 12.4. Задача 4: миграция на новую схему

Добавьте колонку deleted_at в таблицу posts через систему миграций. Решение:

```
# migrations/006_add_soft_delete_to_posts.sql
ALTER TABLE posts ADD COLUMN deleted_at TEXT;
CREATE INDEX idx_posts_deleted
    ON posts(deleted_at)
    WHERE deleted_at IS NULL;

# Запуск мигратора
php migrator.php
# Применяется: 006_add_soft_delete_to_posts.sql
#   OK
# Все миграции применены
```

## 12.5. Задача 5: класс UserRepository

Создайте класс UserRepository с методами findByEmail, searchByName, countActive. Решение:

```php
<?php
class UserRepository extends Model
{
    protected string $table = 'users';

    public function findByEmail(string $email): ?array
    {
        $stmt = $this->pdo->prepare(
            'SELECT * FROM users WHERE email = ?'
        );
        $stmt->execute([$email]);
        $row = $stmt->fetch();
        return $row === false ? null : $row;
    }

    public function searchByName(string $q): array
    {
        $stmt = $this->pdo->prepare(
            'SELECT * FROM users
             WHERE name LIKE ?
             ORDER BY name'
        );
        $stmt->execute(["%$q%"]);
        return $stmt->fetchAll();
    }

    public function countActive(): int
    {
        return (int) $this->pdo->query(
            'SELECT COUNT(*) FROM users
             WHERE deleted_at IS NULL'
        )->fetchColumn();
    }
}
?>
```

## 12.6. Задача 6: регистрация пользователя в транзакции

Создайте пользователя с профилем в одной транзакции. Если что-то одно не получилось — отменить всё. Решение:

```php
<?php
$pdo = db();
try {
    $pdo->beginTransaction();

    # Создать пользователя
    $pdo->prepare(
        'INSERT INTO users (name, email, password_hash)
         VALUES (?, ?, ?)'
    )->execute([
        $name,
        $email,
        password_hash($password, PASSWORD_DEFAULT),
    ]);
    $userId = $pdo->lastInsertId();

    # Создать профиль
    $pdo->prepare(
        'INSERT INTO profiles
            (user_id, bio, avatar_url)
         VALUES (?, ?, ?)'
    )->execute([$userId, '', '']);

    $pdo->commit();
    echo "Регистрация успешна, ваш id: $userId";
} catch (PDOException $e) {
    $pdo->rollBack();
    if ($e->getCode() === '23000') {
        echo 'Email уже зарегистрирован';
    } else {
        error_log($e->getMessage());
        echo 'Ошибка регистрации';
    }
}
?>
```

## 12.7. Задача 7: мини-блог с пагинацией

Выведите посты блога с пагинацией: 5 постов на странице, навигация внизу.
Решение:

```php
<?php
require_once 'db.php';

$perPage = 5;
$page = max(1, (int)($_GET['page'] ?? 1));
$offset = ($page - 1) * $perPage;

$pdo = db();
$total = $pdo->query('SELECT COUNT(*) FROM posts')
             ->fetchColumn();
$pages = max(1, (int) ceil($total / $perPage));

$stmt = $pdo->prepare(
    'SELECT posts.*, users.name AS author
     FROM posts
     INNER JOIN users ON posts.user_id = users.id
     ORDER BY posts.created DESC
     LIMIT ? OFFSET ?'
);
# LIMIT/OFFSET нужно биндить как int
$stmt->bindValue(1, $perPage,  PDO::PARAM_INT);
$stmt->bindValue(2, $offset, PDO::PARAM_INT);
$stmt->execute();
$posts = $stmt->fetchAll();
?>

<?php foreach ($posts as $p): ?>
  <article>
    <h2><?= htmlspecialchars($p['title']) ?></h2>
    <small><?= $p['author'] ?>, <?= $p['created'] ?></small>
    <p><?= nl2br(htmlspecialchars(
            mb_substr($p['body'], 0, 200))) ?>...</p>
  </article>
<?php endforeach; ?>

<nav class="pagination">
  <?php if ($page > 1): ?>
    <a href="?page=<?= $page - 1 ?>">← Назад</a>
  <?php endif; ?>
  <span>Страница <?= $page ?> из <?= $pages ?></span>
  <?php if ($page < $pages): ?>
    <a href="?page=<?= $page + 1 ?>">Вперёд →</a>
  <?php endif; ?>
</nav>
```

## 12.8. Итоги по главам

Глава 1. SQLite — встроенная СУБД без сервера, один файл. Идеальна для обучения, прототипов, малых и средних проектов. Проверка через phpinfo и sqlite3 CLI.

Глава 2. CREATE TABLE с типами INTEGER/TEXT/REAL/BLOB. PRIMARY KEY AUTOINCREMENT, NOT NULL, DEFAULT, UNIQUE, CHECK — основные ограничения.

Глава 3. PDO — стандартный интерфейс. DSN для SQLite вида sqlite:путь. Обязательно: ERRMODE_EXCEPTION и FETCH_ASSOC. Общая функция db() через static.

Глава 4. CRUD: INSERT через prepare/execute, SELECT через fetch/fetchAll/fetchColumn, UPDATE/DELETE с WHERE. lastInsertId для нового id, rowCount для количества изменённых.

Глава 5. Связи 1:1, 1:N, N:M. FOREIGN KEY с ON DELETE CASCADE/SET NULL/RESTRICT. PRAGMA foreign_keys = ON — обязательно.

Глава 6. INNER JOIN (пересечение), LEFT JOIN (все левые), RIGHT JOIN (поменяйте таблицы). GROUP BY с COUNT/SUM/AVG/MAX/MIN, HAVING для фильтра после группировки, GROUP_CONCAT для склейки.

Глава 7. ACID: атомарность, согласованность, изоляция, долговечность. PDO API: beginTransaction, commit, rollBack. Savepoints для частичного отката.

Глава 8. LIMIT/OFFSET для пагинации, COUNT(*) для общего числа. WHERE с операторами сравнения, BETWEEN, IN, LIKE. ORDER BY с проверкой по белому списку.

Глава 9. SQL-инъекции — подготовленные запросы как защита. Идентификаторы нельзя параметризовать, только белый список. Файл базы вне public/, пароли через password_hash.

Глава 10. ООП-обёртки: класс Database (Singleton) для PDO, абстрактный Model с find/findAll/delete, конкретные User, Post, Comment с переиспользованием.

Глава 11. Миграции: файлы в migrations/, таблица schema_migrations для отслеживания. Migrator в транзакции применяет только новые миграции.

Глава 12. Семь практических задач показали, как combine теорию: гостевая книга, импорт CSV, экспорт JSON, миграция схемы, репозиторий, регистрация в транзакции, мини-блог с пагинацией.

---

## 12.9. Что читать дальше

После освоения этого учебника логично двигаться в трёх направлениях. Во-первых, переход на MySQL/MariaDB — стандарт для production. API PDO тот же, отличается только DSN, типы данных и поддержка RIGHT JOIN. Изучите индексы, EXPLAIN, оптимизацию.

Во-вторых, фреймворки — Laravel (Eloquent ORM), Symfony (Doctrine). Они решают за вас миграции, репозитории, отношения, кеширование запросов. Но понимать□□ную работу PDO полезно даже с фреймворком.

В-третьих, теория реляционных баз: нормализация (1НФ, 2НФ, 3НФ), денормализация для производительности, CAP-теорема для распределённых систем, ACID vs BASE.

Главный совет: не пытайтесь заучить все SQL-операторы. Базы данных — это практика. Спроектируйте свою базу для реального проекта: блог с тегами, интернет-магазин с заказами, таск-трекер. С каждой новой таблицей приходит интуитивное понимание, когда нужен JOIN, а когда подзапрос, когда индекс, а когда денормализация. Опыт складывается из десятков спроектированных и переделанных схем, а не из прочитанных справочников.

---